3.9 先行进位加法器的硬件逻辑实现

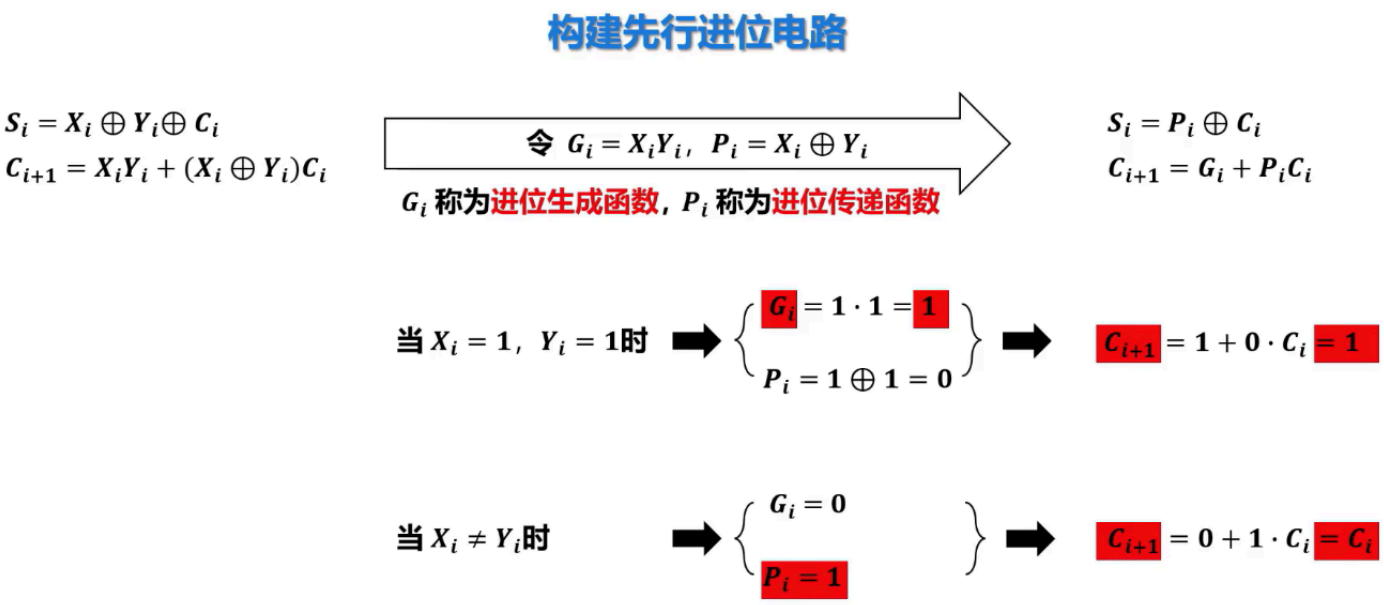
考纲定位：计算机组成原理 · 第 3 章 运算方法（3.9）｜重要度 ★★★★★｜常见考法：大题（G/P、进位展开、4 位 CLA、16 位两级、延迟对比）

一句话核心：用“生成 G”和“传播 P”提前算出所有进位，打破串行进位的逐级等待；组内并行、组间再并行

一、先行进位的构建思路

1.1 考虑打破进位依赖

1. ***约定符号***



定义

- $G_i = A_iB_i$：本位生成进位
- $P_i = A_i \oplus B_i$：本位传播进位
- 有的教材用 $P_i = A_i + B_i$；408 题目会明确，按题设

2. ***写出递归表达式***

构建先行进位电路

$S_i = P_i \oplus C_i$

$C_{i+1} = G_i + P_i C_i$

G_i 称为进位生成函数，　P_i 称为进位传递函数

令 $G_i = X_i Y_i$，　$P_i = X_i \oplus Y_i$

$S_i = X_i \oplus Y_i \oplus C_i$

$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0) = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3(G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0) = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

$$\vdots$$

$$C_n = G_{n-1} + P_{n-1} G_{n-2} + P_{n-1} P_{n-2} G_{n-3} + \cdots + P_{n-1} P_{n-2} \cdots P_1 P_0 C_0$$

向高位的进位输出C_n由 G_i 、 P_i以及C_0经过逻辑运算得到， C_n不再依赖于C_{n-1}。

$$C_{i+1} = G_i + P_i C_i$$

展开

- $C_1 = G_0 + P_0 C_0$
- $C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$
- $C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$
- $C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$

关键

- 每个 C_i 只依赖输入 G/P 和 C_0，不需要等前一级进位
- 代价是逻辑门数量增加

3. ***画出组合电路图***

$C_{i+1} = G_i + P_i C_i$　　令 $G_i = X_i Y_i$，　$P_i = X_i \oplus Y_i$

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$

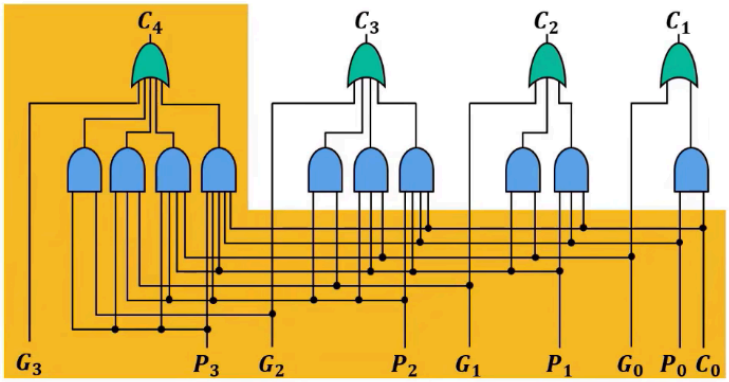
$$C_3 = G_2 + P_2 C_2 = G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0) = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

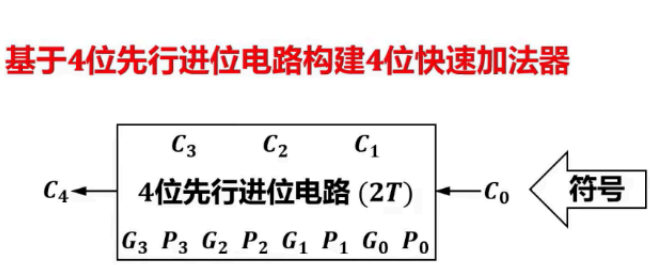
$$C_4 = G_3 + P_3 C_3 = G_3 + P_3(G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0) = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

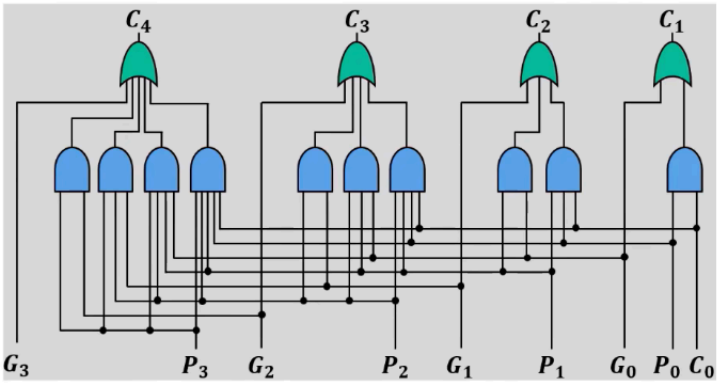
画出相应的组合逻辑电路

$$\vdots$$

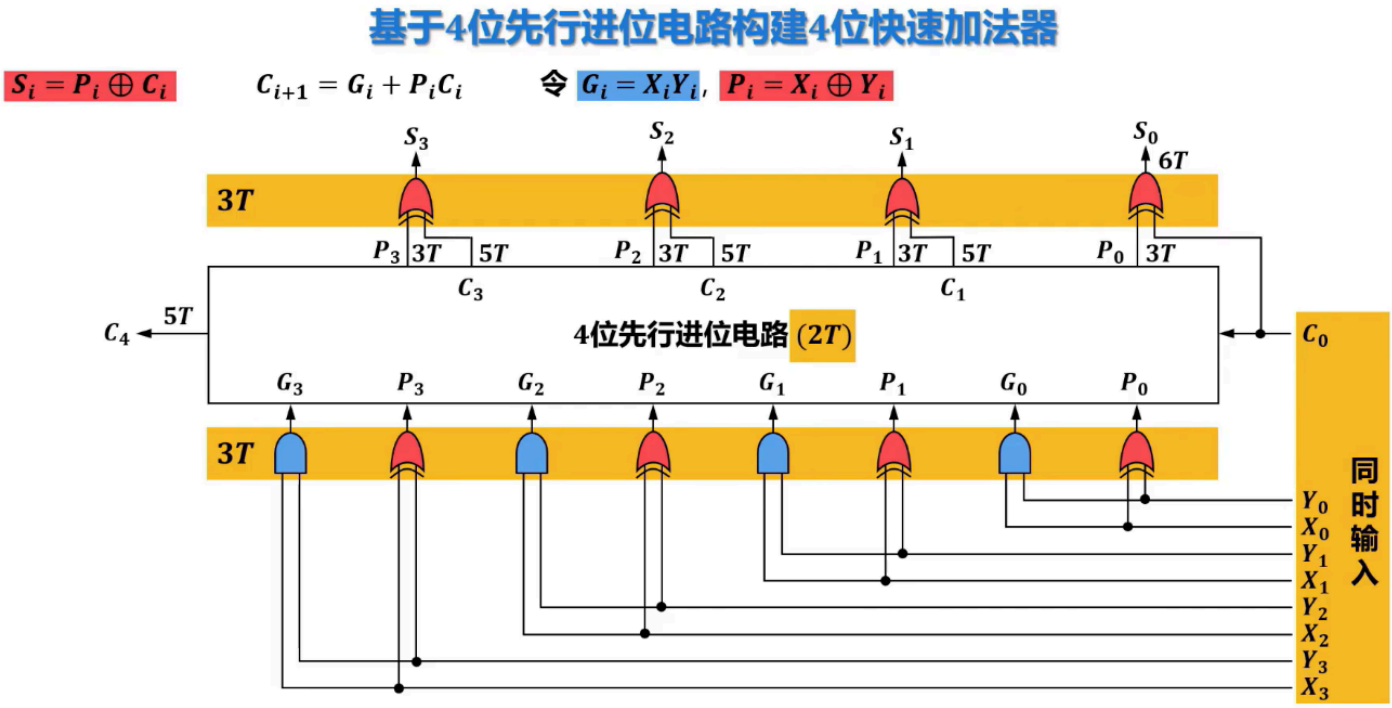
$$C_n = G_{n-1} + P_{n-1} G_{n-2} + P_{n-1} P_{n-2} G_{n-3} + \cdots + P_{n-1} P_{n-2} \cdots P_1 P_0 C_0$$







4. ***四位快速加法器电路图***

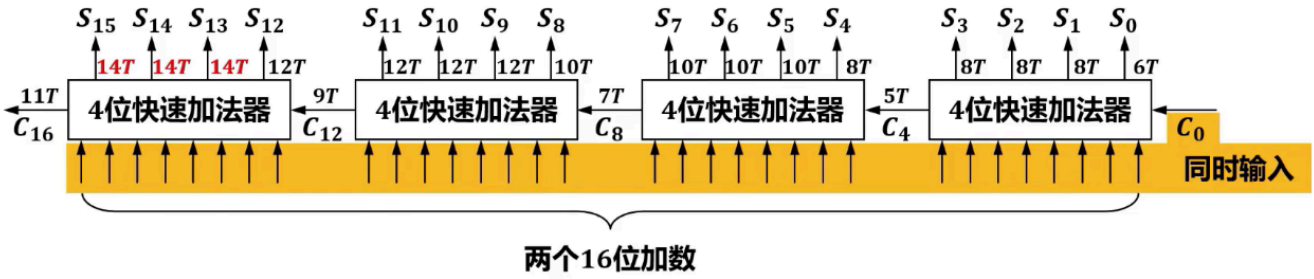


4 位 CLA 结构

- 4 个全加器负责求和
- 先行进位电路同时产生 $C_1 \sim C_4$
- 所有进位由 G_i, P_i, C_0 直接算出

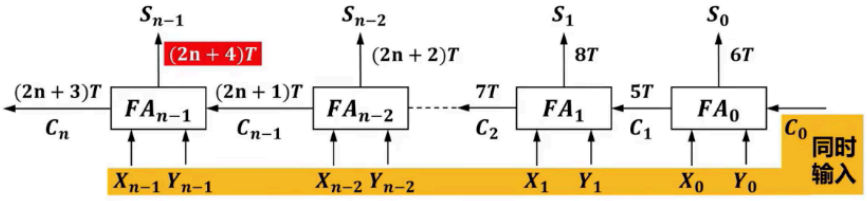
5. ***快速加法器与串行加法器性能对比***

使用4位快速加法器构建更大位宽的加法器电路



上述加法器电路的时间关键延迟为： $14T$

S_{n-1}的时间延迟为: $(2\times 16+4)T=36T$



对比

- 串行：进位逐级传，延迟随位数线性增加
- 先行：进位并行算，延迟随位数对数增加
- 先行进位用更多门换更短延迟

易错

- P_i 的定义有两种：$A_i \oplus B_i$ 或 $A_i + B_i$；代入公式时不要混用
- G_i 为 1 表示“本位一定产生进位”，与低位无关
- P_i 为 1 表示“低位有进位就传上去”，不是“本位一定进位”

二、基于基础先行进位加法器的优化

相比串行进位器，上图实现了组内并行，但组间依旧串行；进一步把组间也并行，需要组生成/组传播函数

2.1 符号约定

可级联的4位先行进位电路

$C_{i+1} = G_i + P_i C_i$　　令 $G_i = X_i Y_i$，$P_i = X_i \oplus Y_i$

$C_1 = G_0 + P_0 C_0$

$C_2 = G_1 + P_1 C_1 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$

$C_3 = G_2 + P_2 C_2 = G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0) = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

$C_4 = G_3 + P_3 C_3 = G_3 + P_3(G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0) = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$

$\vdots$

$C_n = G_{n-1} + P_{n-1} G_{n-2} + P_{n-1} P_{n-2} G_{n-3} + \cdots + P_{n-1} P_{n-2} \cdots P_1 P_0 C_0$

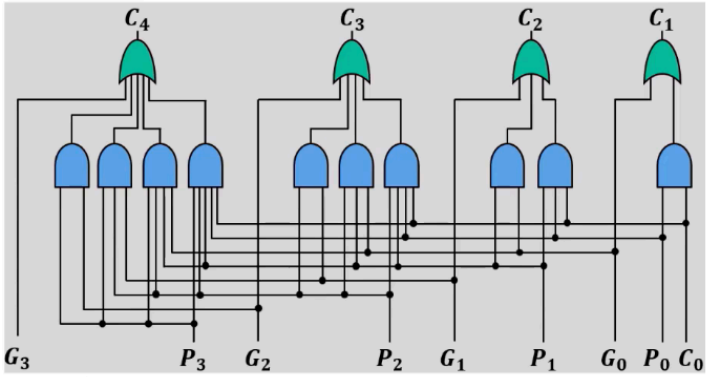
画出相应的组合逻辑电路

$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$

记为 G^*　　记为 P^*

形式相同

$C_4 = G^* + P^* C_0$



$G^* = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0$　组进位生成函数

$P^* = P_3 P_2 P_1 P_0$　组进位传递函数

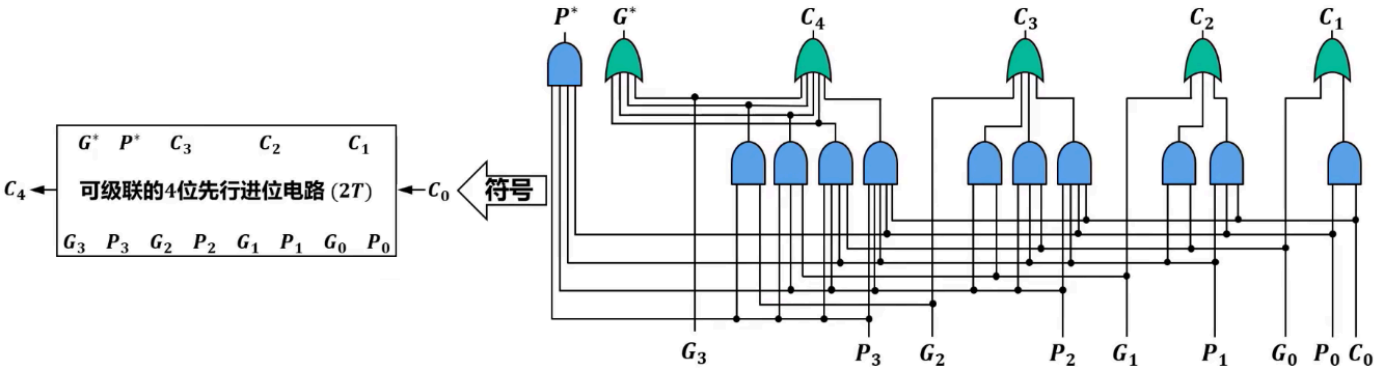
2.2 添加了组进位生成函数和组进位传递函数

可级联的4位先行进位电路

$G^* = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0$　组进位生成函数

$P^* = P_3 P_2 P_1 P_0$　组进位传递函数

添加到4位先行进位电路



4 位组

- $G^* = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0$
- $P^* = P_3 P_2 P_1 P_0$

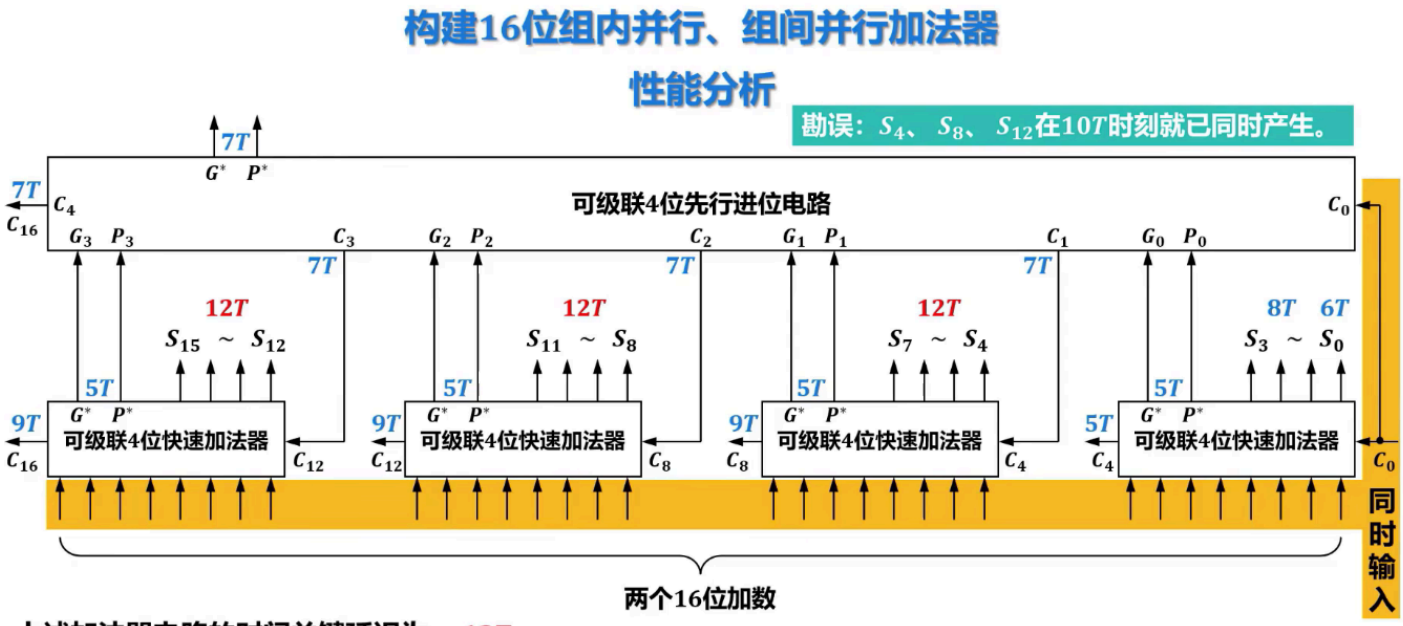
组进位

- $C_4 = G^* + P^* C_0$
- 组间进位由各组的 G^*, P^* 再并行计算

易错

- G^* 不是四个 G 的简单或；要按“高位能否收到低位进位”展开
- P^* 是四个 P 的与，表示整组能否传播进位
- 组间先行进位电路只算组间进位，不重复算组内进位

2.3 十六位组间与组内皆并行的加法器



上述加法器电路的时间关键延迟为： $12T$

两级结构

- 第一级：4 个 4 位 CLA，组内并行
- 第二级：组间先行进位电路，用 4 组 G^*, P^* 算 C_4, C_8, C_{12}, C_{16}
- 各组的和再由组内进位与组间进位共同决定

延迟

- 4 位组内先行：约 2~3 级门延迟
- 16 位两级：约 4 级门延迟
- 位数越多，两级/多级结构的优势越明显

易错

- 16 位不是“16 个全加器串起来”，而是 4 个 4 位组 + 组间先行
- 组间进位的编号与组内进位编号不同，但物理上是同一根线
- 组内先行电路仍需 C_0；组间电路只负责 C_4, C_8, C_{12}, C_{16}

三、一句话总结

先行进位 = 用 G/P 把“进位递推”改写成“进位展开”，再用分组把展开规模控制在可实现范围内

- 组内：4 位 CLA
- 组间：组 G^*/P^* + 第二级 CLA
- 延迟：从 $O(n)$ 降到 $O(\log n)$

知识关联：全加器（3.7 一位全加器的硬件逻辑实现）；串行进位（3.8 串行进位加法器的硬件逻辑实现）；逻辑门（3.6 定点数的加减运算——逻辑代数与逻辑门）；补码加减（3.4 定点数的加减运算——补码运算规则）